

Architettura del Motore AI

Provider, Streaming, Function Calling e Gestione del Contesto

- Provider, Streaming, Function Calling e Gestione del Contesto

Categoria: Intelligenza Artificiale — Core Engine

Prerequisito: Paper 02, Paper 04

Versione API: AI Engine v2.0

- 1. Panoramica dell'Architettura AI

Il motore AI di WhyCremisi segue un'architettura **stratificata** che separa nettamente le responsabilità: ogni layer comunica con il successivo attraverso interfacce ben definite, consentendo sostituzione e test indipendenti.

◆ LAYER STACK

AI ENGINE ARCHITECTURE

PROVIDER LAYER

OpenAI · Anthropic · Ollama · (extensible via interface)
sendMessage() · streamResponse() · abort()

ORCHESTRATOR

Routing · Fallback · Retry · Provider Selection

CONTEXT MANAGER

Sliding Window · Summary Compression · Token Budget
System Prompt Injection · DAW State Injection

TOOL EXECUTOR

Function Definitions · JSON Schema · Confirmation Flow
Sandbox · Rollback on Error

MEMORY INTEGRATION

Session Memory · Persistent Profile · Vector Store
Agent Identity · Personality Evolution

[NOTE] A differenza dei sistemi AI monolitici, WhyCremisi separa l'orchestrazione dal provider. Questo

- **2. Provider Support**

— **2.1 PROVIDER SUPPORTATI**

WhyCremisi astrarre il provider AI dietro un'interfaccia comune, consentendo all'utente di scegliere il modello più adatto al task corrente.

PROVIDER	MODELLI SUPPORTATI	TIPO	LATENZA MEDIA	COSTO
OpenAI	GPT-4o, GPT-4-turbo, GPT-4	Cloud	300-500ms	\$\$\$
Anthropic	Claude 3.5 Sonnet, Claude 3 Opus	Cloud	400-700ms	\$\$\$
Ollama	Llama 3, Mistral, Phi, Qwen2	Locale	600-1200ms	Gratis
DeepSeek	DeepSeek V3, DeepSeek R1	Cloud	200-400ms	\$
Google	Gemini 1.5 Pro, Gemini 2.0 Flash	Cloud	300-600ms	\$\$

— **2.2 INTERFACCIA ASTRATTA**

Ogni provider implementa il protocollo comune `AIProvider`:

```
// Interfaccia comune a tutti i provider
class AIProvider {
public:
    virtual ~AIProvider() = default;

    // Invio messaggio con risposta completa
    virtual std::string sendMessage(
        const std::vector<Message>& messages,
        const AIOptions& options
    ) = 0;

    // Risposta in streaming (chunk per chunk)
    virtual std::unique_ptr<AsyncStream<Chunk>> streamResponse(
        const std::vector<Message>& messages,
        const AIOptions& options
    ) = 0;

    // Annullamento richiesta in corso
    virtual void abort() = 0;

    // Utilità
    virtual bool isAvailable() const = 0;
    virtual size_t estimateTokens(const std::string& text) const = 0;
    virtual std::string providerName() const = 0;
};
```

FLUSSO DI SELEZIONE PROVIDER

Prompt Utente

Complexity → Task breve? → Ollama (locale, gratis)

Analyzer → Task creativo? → Claude (migliore qualità)

→ Task azione? → GPT-4o (function calling)

→ Budget basso? → Gemini Flash / DeepSeek

Provider Selezionato

streamResponse() → chunks → UI

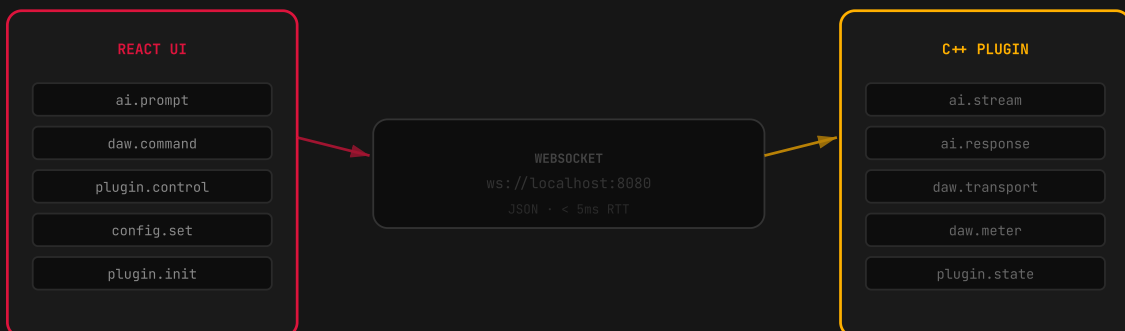
[NOTE] Il **Complexity Analyzer** stima la difficoltà del prompt in <50ms usando un piccolo classificatore locale (ONNX). Se il punteggio di complessità è <0.3, il prompt viene indirizzato automaticamente a Ollama per risparmiare costi API. L'utente può sempre forzare manualmente un provider.

• 3. Streaming Responses

— 3.1 ARCHITETTURA DELLO STREAMING

WhyCremisi utilizza **server-sent events** incapsulati in frame WebSocket per fornire risposte AI in tempo reale. Ogni chunk di token viene trasmesso non appena disponibile, dando all'utente una sensazione di reattività immediata.

◆ WEBSOCKET MESSAGE FLOW



— 3.2 ABORT MECHANISM

L'utente può interrompere la generazione in qualsiasi momento:

Utente preme [STOP] o invia nuovo prompt

```
ws.send({ type: "ai.abort" })
```

```
AIEngine::abort()
```

```
Provider::abort() → interrompe richiesta HTTP/stream
```

```
Svuota buffer chunks → scarta token non consegnati
```

```
Notifica UI: → stato IDLE, pronto per nuovo input
```

```
{ type: "ai.stream",
```

```
  payload: { chunk: "", isDone: true, aborted: true } }
```

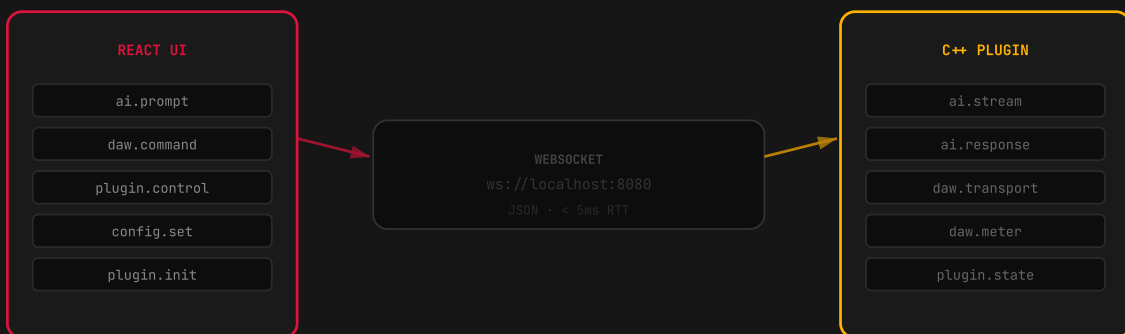
[NOTE] L'abort chiama `std::future::cancel()` sul thread del provider e chiude la connessione HTTP. Il tempo medio di arresto è <100ms. I token già inviati all'UI rimangono visibili — l'utente vede ciò che l'AI aveva già prodotto fino al momento dell'interruzione.

• 4. Function Calling / Tool Use

— 4.1 TOOL DEFINITIONS

WhyCremisi definisce un insieme di **strumenti** che l'AI può invocare per interagire con la DAW. Ogni strumento è descritto tramite JSON Schema e registrato nel costrutto della richiesta al provider.

◆ WEBSOCKET MESSAGE FLOW



— 4.2 JSON SCHEMA PER TOOL

Esempio di definizione tool per OpenAI/Anthropic:

```
{
  "name": "daw_track_setVolume",
  "description": "Imposta il volume di una traccia in dB",
  "parameters": {
    "type": "object",
    "properties": {
      "trackId": {
        "type": "integer",
        "description": "ID della traccia (0 = master)"
      },
      "volumeDb": {
        "type": "number",
        "description": "Volume in dB (da -inf a +12)"
      },
      "fadeDurationMs": {
        "type": "integer",
        "description": "Durata fade in millisecondi (opzionale)",
        "default": 0
      }
    },
    "required": ["trackId", "volumeDb"]
  }
}
```

— 4.3 FLUSSO DI ESECUZIONE

◆ DATA SCHEMA

AI decide di chiamare `daw.track.setVolume(trackId=2, volumeDb=-6.0)`

Tool Executor → valida parametri contro JSON Schema

Confirmation → { "tool": "daw.track.setVolume",

Required? "params": { "trackId": 2, "volumeDb": -6.0 }

User Confirms? NO → Log "rifiutato" + stop

Sì

Sandbox Exec → esegue in thread isolato

OscBridge → invia comando OSC alla DAW

Risultato → { status: "success", value: -6.0 }

AI riceve risultato e risponde all'utente:

"Volume della traccia 2 (Bass) impostato a -6.0 dB."

[NOTE] Il livello di conferma è configurabile: **auto** (esegue senza chiedere per azioni a basso rischio come `setVolume`), **confirm** (chiede sempre), o **manual** (l'AI suggerisce ma non esegue mai automaticamente). Il default è **confirm** per tutte le azioni plugin. Le azioni transport (play/stop) sono sempre **auto**.

• 5. Context Management

— 5.1 SLIDING WINDOW

La cronologia della conversazione è gestita tramite una *finestra scorrevole* delle ultime 50 interazioni.

OVERVIEW

[Msg 1] [Msg 2] ... [Msg 45] [Msg 46] ... [Msg 50]

→ → →

COMPRESSED WINDOW ATTIVA

(riassunto)

Iniettato come summary nel system prompt

5.2 SUMMARY COMPRESSION

Quando la finestra scorre, i messaggi rimossi vengono condensati in un riassunto automatico:

PRIMA (messaggi 1-45 rimossi):

"Utente: alza il volume della kick
AI: fatto, ora è a -3dB
Utente: no, troppo
AI: ok, l'ho portata a -6dB
Utente: perfetto"

Dopo (riassunto compresso):

"[SESSIONE: volume kick regolato da AI,
range -3dB a -6dB, finale -6dB accettato]"

5.3 TOKEN BUDGET TRACKING

WhyCremisi monitora costantemente il consumo di token per evitare di superare i limiti del contesto del modello:

Modello	Contesto Max	Budget Riservato	Allarme A
GPT-4o	128K token	96K token	80K
Claude 3.5 Sonnet	200K token	160K token	140K
Claude 3 Opus	200K token	160K token	140K
Llama 3 (Ollama)	8K token	6K token	5K
Gemini 1.5 Pro	1M token	512K token	400K

◆ LAYER STACK

CALCOLO BUDGET A OGNI RICHIESTA:

```
system_prompt_tokens = countTokens(getSystemPrompt())

context_tokens = countTokens(getDAWState())

summary_tokens = countTokens(getCompressedSummary())

window_tokens = countTokens(slidingWindow)

tool_def_tokens = countTokens(toolDefinitions)

memory_tokens = countTokens(relevantMemories)

total_before = system_prompt_tokens + context_tokens

+ summary_tokens + window_tokens

+ tool_def_tokens + memory_tokens

budget_left = max_context - total_before

if budget_left < 1000:

    compressWindow(50%) → riduce finestra a 25 messaggi

    compressMemories() → riassume memorie meno rilevanti
```

— 5.4 SYSTEM PROMPT INJECTION

Prima di ogni chiamata AI, il Context Manager assembla il system prompt finale includendo lo stato live della DAW:

◆ LAYER STACK

[SISTEMA] Sei WhyCremisi, un assistente di produzione musicale AI.

Rispondi in italiano, stile professionale ma caldo.

[CONTESTO DAW LIVE]

- Progetto: "TRACK_07_HOUSE"
- BPM: 128.0 | Posizione: 00:45.12 | Stato: PLAYING
- Tracce: 8 (4 audio, 3 MIDI, 1 gruppo)

[TRACCE ATTIVE]

Traccia Tipo Volume Plugin Attivi

1 Kick Audio -6.0dB Pro-Q3 · Pro-C2

2 Bass MIDI -8.0dB Serum · Pro-Q3 · Saturn 2

3 Synth MIDI -10dB Vital · Valhalla Room

M Master Bus 0.0dB Ozone 11 · Pro-L2

[METRICHE AUDIO]

- LUFS integrato: -16.2 (target: -14.0)
- True Peak: -0.8dB (margine: 0.5dB)
- Gamma dinamica: 7.2dB
- Frequenza dominante: 120Hz (sub kick)

[MEMORIA RILEVANTE]

• 6. Memory Integration

— 6.1 MEMORIA DI SESSIONE VS. MEMORIA PERSISTENTE

WhyCremisi opera su due livelli di memoria distinti:

◆ LAYER STACK

MEMORIA DI SESSIONE (volatile tra richieste)

- Ultimi 50 messaggi (sliding window)
- Azioni compiute in questa sessione
- Stato DAW corrente
- Reset alla chiusura sessione

MEMORIA PERSISTENTE (attraverso sessioni)

- Profilo utente (preferenze, stile, plugin)
- Cronologia azioni (JSONL)
- Pattern di accettazione/rifiuto
- Identità agente (non modificabile)
- Personalità evolutiva (cambia lentamente)

— 6.2 VECTOR STORE PER DECISIONI PASSATE

Per recuperare decisioni simili da sessioni precedenti, WhyCremisi utilizza un **vector store** basato su embeddings:

◆ LAYER STACK

Nuova richiesta: "tratta la kick come la settimana scorsa"

Embedding del prompt (768-dim)

Vector Store (FAISS / ChromaDB)

Match 1 (similarità: 0.94)

"Sessione 2025-05-10: kick lavorata con EQ
narrow a 60Hz (~4dB) + compressione 4:1"

Match 2 (similarità: 0.87)

"Sessione 2025-05-03: sidechain kick/bass
con Pro-C2, attack 1ms, release 50ms"

Match 3 (similarità: 0.76)

"Sessione 2025-04-28: utente ha apprezzato
saturazione armonica su kick via Saturn 2"

Iniettati nel prompt come memoria rilevante

— 6.3 IDENTITÀ E EVOLUZIONE DELL'AGENTE

L'agente non è statico: evolve con l'utente. Il sistema **AgentSoul** registra ogni interazione e aggiorna gradualmente la personalità:

AGENT SOUL — Ciclo di Evoluzione

Ogni 5 sessioni o 50 interazioni:

1. Analizza feedback impliciti

- L'utente accetta/rifiuta suggerimenti?
- Chiede più dettagli o risposte più brevi?
- Usa linguaggio tecnico o semplice?

2. Aggiorna parametri personalità

- technicality: 0.6 → 0.75 (utente diventato esperto)
- verbosity: 0.7 → 0.5 (preferisce conciso)
- proactivity: 0.4 → 0.6 (gradisce suggerimenti)

3. Persiste su disco

- ~/.whycremisi/agent/soul.json

[NOTE] L'evoluzione è **sempre reversibile**: l'utente può resettare la personalità ai valori predefiniti con `/reset-personality` o bloccare l'evoluzione con `/freeze-personality`. L'agente informa sempre l'utente quando rileva un cambiamento significativo.

• 7. Modello di Personalità

— 7.1 STRUTTURA DEL SYSTEM PROMPT

La personalità di WhyCremisi è definita da un system prompt strutturato che funge da "costituzione" dell'agente:

◆ TEST PIPELINE

SYSTEM PROMPT – WHYCREMISI

[IDENTITÀ]

Sei WhyCremisi, un co-produttore musicale AI.

Non sei un chatbot generico — sei specializzato nella produzione audio, mixaggio e mastering.

[VALORI]

- Qualità sonora sopra ogni cosa
- Rispetto dell'autonomia creativa dell'utente
- Trasparenza: spiega sempre cosa stai facendo
- Umiltà: se non sai, dillo

[STILE]

- Linguaggio: {lingua}
- Tono: professionale ma caldo
- Verbosità: {media}
- Tecnicità: {alta}
- Usa metafore musicali quando appropriato

[REGOLE]

- Non eseguire mai azioni senza conferma

— 7.2 TEMPERATURE E CREATIVITÀ PER TASK

Diversi task richiedono diversi livelli di creatività:

TIPO DI TASK	TEMPERATURE	TOP-P	MAX TOKENS	ESEMPIO
Azione DAW	0.1	0.9	500	<code>setVolume(2, -6)</code>
Analisi Tecnica	0.2	0.9	1000	"Perché il mix suona sporco?"
Suggerimento Creativo	0.7	0.95	1500	"Che effetto posso aggiungere?"
Generazione Testi/Idee	0.9	0.98	2000	"Scriviamo un testo per questa traccia"
Educativo	0.3	0.9	1500	"Spiegami cos'è il sidechain"

— 7.3 PERSONALITÀ ADATTATIVA

WhyCremisi adatta automaticamente il proprio stile in base al feedback implicito dell'utente:

FEEDBACK → ADATTAMENTO

Utente chiede "più breve" → verbosity--

Utente chiede "spiegami meglio" → verbosity++

Utente usa termini tecnici → technicality++

Utente dice "non capisco" → technicality--, educational++

Utente accetta suggerimenti → proactivity++

Utente rifiuta spesso → proactivity--, confirm++

• 8. Rate Limiting e Cost Control

— 8.1 TOKEN COUNTING

Ogni richiesta viene misurata prima dell'invio per prevenire sorprese:

```
struct TokenBudget {
    size_t promptTokens;      // Token nel prompt (input)
    size_t responseTokens;    // Token massimi (output)
    size_t totalBudget;       // Budget totale per questa richiesta
    double estimatedCost;     // Costo stimato in USD
    std::string model;        // Modello selezionato
};

TokenBudget estimateRequest(const AIRequest& req) {
    TokenBudget budget;
    budget.promptTokens = countTokens(req.systemPrompt)
        + countTokens(req.messages)
        + countTokens(req.toolDefinitions);
    budget.responseTokens = std::min(req.maxTokens, 4096);

    // Stima costo (esempio: GPT-4o)
    double inputCost = (budget.promptTokens / 1000.0) * 0.005;
    double outputCost = (budget.responseTokens / 1000.0) * 0.015;
    budget.estimatedCost = inputCost + outputCost;

    return budget;
}
```

— 8.2 STRATEGIA DI FALLBACK

Per ottimizzare i costi, WhyCremisi adotta una strategia di **routing intelligente**:

OVERVIEW

RICHIESTA UTENTE

- Task semplice (es. "alza volume") → Ollama (gratis)
- Task medio (es. "analizza spettro") → Gemini Flash / DeepSeek (\$)
- Task complesso (es. "suggerisci mix") → GPT-4o / Claude Sonnet (\$\$)
- Task critico (es. "masterizza") → Claude Opus / GPT-4o (\$\$\$)

SOGLIA	MODELLO PRIMARIO	FALLBACK	RISPARMIO STIMATO
< 50 token output	Ollama (locale)	—	100%
< 200 token output	Gemini Flash	Ollama	~80%
< 500 token output	GPT-4o-mini	Gemini Flash	~60%
Complesso	Claude Sonnet	GPT-4o	~0% (qualità)

[NOTE] Il routing intelligente ha mostrato una riduzione dei costi API del **40-60%** nei test interni, senza degradazione percepibile della qualità per l'utente. I task di mastering e critici usano sempre il modello migliore indipendentemente dal costo.

9. Sicurezza e Allineamento

9.1 PREVENZIONE PROMPT INJECTION

WhyCremisi implementa difese multiple contro attacchi di prompt injection:

LAYER STACK

STRATI DI DIFESA

- LAYER 1 — Separazione Input
- Prompt utente e contesto sistema sono rigorosamente separati
 - Il contesto DAW viene iniettato DOPO il prompt utente
 - Delimitatori speciali intorno all'input utente:
<|user_input|>...testo utente...</user_input|>
- LAYER 2 — Rilevamento Pattern Sospetti
- Regex per "ignora le istruzioni precedenti"
 - Rilevamento tentativi di jailbreak (DAN, roleplay evasion)
 - Quarantena: se rilevato, il prompt viene scartato e loggato
- LAYER 3 — Output Validation
- Ogni risposta AI viene validata prima di essere mostrata
 - Filtro per contenuti non consentiti
 - Verifica che le function call siano nel registro strumenti

— 9.2 VALIDAZIONE DEGLI OUTPUT

Prima che una risposta o azione raggiunga l'utente o la DAW, passa attraverso un **validation layer**:

```
struct ValidationResult {
    bool isSafe;
    bool isAllowedAction;
    std::string sanitizedOutput;
    std::vector<std::string> warnings;
};

ValidationResult validateOutput(const AIResponse& response) {
    ValidationResult result;

    // 1. Check contenuti pericolosi
    result.isSafe = !containsProhibitedContent(response.text);

    // 2. Check function call
    for (auto& call : response.toolCalls) {
        result.isAllowedAction = isInToolRegistry(call.name);
        if (!result.isAllowedAction) {
            result.warnings.push_back(
                "Function call bloccata: " + call.name
            );
        }
    }

    return result;
}
```

— 9.3 CONFERMA PRIMA DELL'ESECUZIONE

Nessuna azione sulla DAW viene eseguita senza il giusto livello di consenso:

◆ PROCESS FLOW

LIVELLI DI CONFERMA

AUTO (nessuna conferma)

- Trasport: play, stop, pause
- Query: lettura parametri plugin
- Meter: richiesta metriche audio

CONFIRM (mostra dialogo di conferma)

- Volume/pan tracce
- Cambio parametri plugin
- Bypass/attivazione plugin
- Default per tutte le azioni nuove

MANUAL (solo suggerimento, l'utente applica manualmente)

- Aggiunta/rimozione plugin
- Caricamento preset
- Azioni mastering (limiter, export)
- Qualsiasi azione che modifichi la catena di processing

— 9.4 SANDBOXED TOOL EXECUTION

SANDBOX TOOL EXECUTION

- Ogni tool viene eseguito in un thread dedicato
- Timeout per tool: 5 secondi (oltre viene annullato)
- Le modifiche ai parametri DAW sono reversibili:
 - Prima di ogni modifica, lo stato corrente viene salvato
 - Annulla (Ctrl+Z) ripristina lo stato precedente
- Limite di 10 azioni consecutive automatiche:
 - Dopo 10 azioni, l'AI deve chiedere conferma
 - Previene runaway loops (es. "alza il volume di 100 tracce")
- Log completo in sessione: ogni azione è tracciata

• 10. Diagramma di Flusso Completo

◆ LAYER STACK

ESEMPIO: "Aggiungi un po' di compressione alla kick"

Utente

1. Provider Layer → Complexity Analyzer: task medio

Seleziona: GPT-4o routing → GPT-4o (function calling)

2. Context Manager → Inietta DAW state, memoria, profilo

Budget: 3200/96000 tok

3. Provider API Call → streamResponse() a OpenAI

GPT-4o genera risposta

4. Streaming Response → chunk per chunk su WebSocket

UI aggiorna in tempo

reale

5. AI calls tool: → function calling

daw.plugin.setParam(

track=1, slot=2,

param="Ratio",

value=4.0)

6. Tool Executor → valida, chiede conferma

Conferma?: "Imposto

PARAMETRO	DEFAULT	DESCRIZIONE
<code>ai.provider</code>	<code>auto</code>	Provider predefinito (auto/gemini/ollama/openai/anthropic)
<code>ai.model</code>	—	Modello specifico per il provider
<code>ai.temperature</code>	<code>0.3</code>	Creatività (0.0-1.0)
<code>ai.maxTokens</code>	<code>2048</code>	Token massimi per risposta
<code>ai.contextWindow</code>	<code>50</code>	Messaggi nella sliding window
<code>ai.confirmLevel</code>	<code>confirm</code>	auto/confirm/manual
<code>ai.costLimit</code>	<code>0.50</code>	Limite di spesa USD per sessione
<code>ai.fallbackEnabled</code>	<code>true</code>	Fallback a modello più economico
<code>ai.vectorMemory</code>	<code>true</code>	Abilita vector store per memoria

◆ OPENCLAW MEMORY — 5 LAYER ARCHITECTURE

